Function Based View → APIView → ViewSet → ModelViewSet

👉 **kivabe data access korba (request, user, params, etc.)**

🧠 0. Common Idea (IMPORTANT)

👉 Django REST e sob view e same jinis access kora jai:

Data	Access
Body data	`request.data`
Query params	`request.query_params`
User	`request.user`
URL param	`pk / kwargs`
Headers	`request.headers`

🔰 1. Function Based View (FBV)

👉 Basic + manual

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from contact.models import Contact
from contact.serialisers import ContactSerializer

@api_view(['GET', 'POST'])
def contact_view(request):

    # 📥 GET data
    if request.method == 'GET':
        contacts = Contact.objects.all()
        serializer = ContactSerializer(contacts, many=True)
        return Response(serializer.data)

    # 📤 POST data
    if request.method == 'POST':
        print(request.data)  # 🔥 data access
```

```
        serializer = ContactSerializer(data=request.data)

        if serializer.is_valid():
            user = request.user if request.user.is_authenticated else None
            serializer.save(user=user)

            return Response(serializer.data)

        return Response(serializer.errors)
```

🔥 Data Access in FBV

```
request.data              # body
request.query_params      # ?id=1
request.user              # logged user
request.headers           # headers
```

⚙️ 2. APIView (Class Based)

👉 More control, cleaner

```
from rest_framework.views import APIView
from rest_framework.response import Response

class ContactAPIView(APIView):

    def get(self, request):
        email = request.query_params.get('email')  # 🔥 query param

        contacts = Contact.objects.all()
        serializer = ContactSerializer(contacts, many=True)

        return Response(serializer.data)

    def post(self, request):
        print(request.data)        # 🔥 body
        print(request.user)        # 🔥 user

        serializer = ContactSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save(user=request.user if
request.user.is_authenticated else None)
```

```
            return Response(serializer.data)

        return Response(serializer.errors)
```

🔥 Data Access in APIView

Same as FBV but inside class:

```
request.data
request.user
request.query_params
request.headers
```

⚡ 3. ViewSet

👉 group multiple actions (list, create, retrieve)

```
from rest_framework.viewsets import ViewSet

class ContactViewSet(ViewSet):

    def list(self, request):
        contacts = Contact.objects.all()
        serializer = ContactSerializer(contacts, many=True)
        return Response(serializer.data)

    def create(self, request):
        print(request.data)  # 🔥 data

        serializer = ContactSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save(user=request.user)
            return Response(serializer.data)

        return Response(serializer.errors)
```

🔥 Data Access in ViewSet

```
request.data
request.user
request.query_params
self.kwargs   # URL param
```

🚀 4. ModelViewSet (MOST USED 🔥)

👉 automatic CRUD + override when needed

```
from rest_framework.viewsets import ModelViewSet

class ContactModelViewSet(ModelViewSet):
    queryset = Contact.objects.all()
    serializer_class = ContactSerializer
```

👉 but data access korte hole override korte hobe 👇

🔥 Important Methods

✅ 1. get_queryset()

```
def get_queryset(self):
    user = self.request.user

    if user.is_staff:
        return Contact.objects.all()

    return Contact.objects.filter(user=user)
```

✅ 2. perform_create()

```
def perform_create(self, serializer):
    print(self.request.data)   # 🔥 body
    print(self.request.user)   # 🔥 user

    serializer.save(user=self.request.user)
```

✅ 3. create() (full control)

```
def create(self, request, *args, **kwargs):
    print(request.data)

    return super().create(request, *args, **kwargs)
```

✅ 4. URL param

```
def retrieve(self, request, *args, **kwargs):
    pk = self.kwargs.get('pk')
    print(pk)
    return super().retrieve(request, *args, **kwargs)
```

🎯 FULL COMPARISON

Feature	FBV	APIView	ViewSet	ModelViewSet
Control	🔥 Full	🔥 Full	Medium	Medium
Code	More	Medium	Less	Very less
CRUD auto	❌	❌	❌	✅
Best for	Simple	Custom	grouped logic	production CRUD

🧠 FINAL UNDERSTANDING

👉 sob jaygay same data access:

```
request.data          # POST/PUT body
request.user          # JWT user
request.query_params  # URL query
request.headers       # headers
self.kwargs           # URL param (class based)
```

---

complete CRUD + data access dekhai ekta example diye (Contact API)

Ami 4 ta style e dekhabo:

👉 FBV
👉 APIView
👉 ViewSet
👉 ModelViewSet

And sab gula te use korbo:
GET, POST, PUT, PATCH, DELETE

🧱 Example Model (same for all)

```
class Contact(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    message = models.TextField()
```

🔰 1. Function Based View (FBV)

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from contact.models import Contact
from contact.serialisers import ContactSerializer

@api_view(['GET', 'POST', 'PUT', 'PATCH', 'DELETE'])
def contact_view(request, pk=None):

    # 🔍 GET (list or single)
    if request.method == 'GET':
        if pk:
            contact = Contact.objects.get(id=pk)
            serializer = ContactSerializer(contact)
        else:
            contacts = Contact.objects.all()
            serializer = ContactSerializer(contacts, many=True)

        return Response(serializer.data)

    # ➕ POST
```

```
    if request.method == 'POST':
        print(request.data)  # 🔥 data access

        serializer = ContactSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # 🔄 PUT (full update)
    if request.method == 'PUT':
        contact = Contact.objects.get(id=pk)

        serializer = ContactSerializer(contact, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # ⚡ PATCH (partial update)
    if request.method == 'PATCH':
        contact = Contact.objects.get(id=pk)

        serializer = ContactSerializer(contact, data=request.data,
partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # ❌ DELETE
    if request.method == 'DELETE':
        contact = Contact.objects.get(id=pk)
        contact.delete()
        return Response({"message": "Deleted"})
```

⚙️ 2. APIView (Class Based)

```
from rest_framework.views import APIView
from rest_framework.response import Response

class ContactAPIView(APIView):
```

```
    # 🔍 GET
    def get(self, request, pk=None):
        if pk:
            contact = Contact.objects.get(id=pk)
            serializer = ContactSerializer(contact)
        else:
            contacts = Contact.objects.all()
            serializer = ContactSerializer(contacts, many=True)

        return Response(serializer.data)

    # ➕ POST
    def post(self, request):
        print(request.data)  # 🔥

        serializer = ContactSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # 🔄 PUT
    def put(self, request, pk):
        contact = Contact.objects.get(id=pk)

        serializer = ContactSerializer(contact, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # ⚡ PATCH
    def patch(self, request, pk):
        contact = Contact.objects.get(id=pk)

        serializer = ContactSerializer(contact, data=request.data,
partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # ❌ DELETE
    def delete(self, request, pk):
        contact = Contact.objects.get(id=pk)
```

```
        contact.delete()
        return Response({"message": "Deleted"})
```

⚡ 3. ViewSet

```
from rest_framework.viewsets import ViewSet
from rest_framework.response import Response

class ContactViewSet(ViewSet):

    # GET list
    def list(self, request):
        contacts = Contact.objects.all()
        serializer = ContactSerializer(contacts, many=True)
        return Response(serializer.data)

    # GET single
    def retrieve(self, request, pk=None):
        contact = Contact.objects.get(id=pk)
        serializer = ContactSerializer(contact)
        return Response(serializer.data)

    # POST
    def create(self, request):
        print(request.data)

        serializer = ContactSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # PUT
    def update(self, request, pk=None):
        contact = Contact.objects.get(id=pk)

        serializer = ContactSerializer(contact, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # PATCH
    def partial_update(self, request, pk=None):
```

```
        contact = Contact.objects.get(id=pk)

        serializer = ContactSerializer(contact, data=request.data,
partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(serializer.errors)

    # DELETE
    def destroy(self, request, pk=None):
        contact = Contact.objects.get(id=pk)
        contact.delete()
        return Response({"message": "Deleted"})
```

🚀 4. ModelViewSet (BEST 🔥)

👉 automatic sob method already ache

```
from rest_framework.viewsets import ModelViewSet

class ContactModelViewSet(ModelViewSet):
    queryset = Contact.objects.all()
    serializer_class = ContactSerializer
```

🔥 Override for data access

```
# POST
def perform_create(self, serializer):
    print(self.request.data)
    serializer.save()

# GET filter
def get_queryset(self):
    email = self.request.query_params.get('email')

    if email:
        return Contact.objects.filter(email=email)

    return Contact.objects.all()

# PUT/PATCH
```

```
def update(self, request, *args, **kwargs):
    print(request.data)
    return super().update(request, *args, **kwargs)

# DELETE
def destroy(self, request, *args, **kwargs):
    print("Deleting...")
    return super().destroy(request, *args, **kwargs)
```

🎯 FINAL SUMMARY

Method	FBV	APIView	ViewSet	ModelViewSet
GET	if block	get()	list/retrieve	auto
POST	if block	post()	create()	auto
PUT	if block	put()	update()	auto
PATCH	if block	patch()	partial_update()	auto
DELETE	if block	delete()	destroy()	auto

🧠 SUPER IMPORTANT

👉 sob jaygay same:

```
request.data          # POST body
request.user          # JWT user
request.query_params  # ?id=1
self.kwargs           # pk
```

🚀 Final Advice

👉 Beginner → FBV
👉 Intermediate → APIView
👉 Production → ModelViewSet 🔥